

Soluții — constantin-gitops (layer business)

Repo `nimigeanconstantinion/ms-gitops` · branch `master` · 2026-07-22 · review read-only

#	Severitate	Fișier	Problemă	Efort
S1	 Critical	<code>argo-apps/app-kong.yaml</code>	Calea <code>valueFiles</code> pointează la un fișier inexistent → Kong nu se sincronizează	1 linie
S2	 Medium	<code>business/rsk/keycloak/realm-rsk.yaml</code>	Secret client în clar, comis în git (<code>spring-api</code>)	refactor mic

S1 — Kong: calea greșită la values (blochează sync-ul)

● CRITICAL valueFiles pointează la un path fără rsk/

argo-apps/app-kong.yaml

SIMPTOM

Kong rămâne `Unknown / OutOfSync` în ArgoCD. La *App Details* → *Manifest* apare o eroare de tipul `rpc error: ... $values/business/data-service/kong/values.yaml: no such file`. Chart-ul Kong nu primește values.

CAUZĂ

La mutarea serviciului sub `business/rsk/...`, au fost actualizate source-urile `ingress` și `declarative` (au acum `rsk/`), dar `valueFiles` a rămas pe structura veche. Verificat cu `ls`: fișierul din manifest nu există; cel real e la `business/rsk/data-service/kong/values.yaml`.

BEFORE

```
helm:
  releaseName: kong
  valueFiles:
    - $values/business/data-service/kong/values.yaml # NU exista
```

AFTER

```
helm:
  releaseName: kong
  valueFiles:
    - $values/business/rsk/data-service/kong/values.yaml
```

De ce merge: celelalte două source-uri din aceeași `Application` (`../kong/ingress` și `../kong/declarative`) au deja prefixul `rsk/` corect. Aliniind și `valueFiles`, ArgoCD rezolvă `$values` → chart-ul Kong primește config-ul → sync verde.

VERIFICARE (READ-ONLY)

```
# fisierul exista pe noua cale
ls business/rsk/data-service/kong/values.yaml
# dupa git push, in cluster
kubectl -n argocd get app kong -o jsonpath='{.status.sync.status}' # -> Synced
```

S2 — spring-api: secret în clar în realm



MEDIUM

Client confidențial cu secret placeholder comis

business/rsk/keycloak/realm-rsk.yaml — client `spring-api`

SIMPTOM

```
- clientId: spring-api
  publicClient: false
  secret: "your-secret-key-here-change-in-production"
```

Secret de client confidențial în clar, în git. La import Keycloak îl ia *literal* — devine parola reală a clientului.

CAUZĂ

Clientul a fost adus dintr-un JSON de export fără a trece secretul prin fluxul de secrete al clusterului.

SOLUȚIE — 2 OPȚIUNI

Variantă	Când	Cum
A. Elimină clientul (recomandat acum)	Dacă <code>spring-api</code> nu e consumat de nimic — <code>data-service</code> folosește <code>data-service-swagger</code> (public)	Scoate blocul <code>spring-api</code> din realm până e chiar necesar
B. Secret din cluster	Dacă backend-ul chiar are nevoie de client confidențial	Pune secretul într-un <code>SealedSecret</code> și referențiază-l — exact pattern-ul folosit pe <code>car-platform</code> pentru <code>oauth2-proxy</code> (secret generat de Keycloak, exportat via Crossplane)

De ce contează: un secret în git rămâne în istoric chiar și după ce-l ștergi. Regula clusterului: materialul sensibil nu stă în clar în repo — trece prin sealed-secrets sau e generat de operator.

Ce e deja rezolvat (nu necesită acțiune)

Item	Stare
F1 — <code>data-service</code> definit de 2 ori	✅ rezolvat (a rămas doar <code>app-data-service.yaml</code>)
Wildcard <code>*</code> în <code>redirectUri</code>	✅ scos — <code>redirect-uri</code> exacte; <code>"+"</code> doar în <code>webOrigins</code> (corect)
PKCE pe clientul swagger	✅ <code>S256</code> activ
<code>message_key</code> logging	✅ <code>"message"</code> consistent (Spring)

Review read-only — codul elevului nu a fost modificat. Ordinea recomandată: S1 întâi (deblochează Kong), apoi S2.